

Sistema de Filtros — Listado de Bandas

Índice

- 1. Descripción general
- 2. Arquitectura y tecnologías implicadas
- 3. Diferencias respecto al sistema de filtros de músicos
- 4. Flujo completo de datos
- 5. Capa de presentación — Twig
- 6. Autocompletado de ciudad — Stimulus + Google Places
- 7. Geolocalización del navegador
- 8. Slider de radio de proximidad
- 9. Distancia en la tarjeta de banda
- 10. Capa de control — BandaController
- 11. Capa de acceso a datos — BandaRepository
- 12. Fórmula de Haversine
- 13. Estructura de parámetros GET
- 14. Casos de uso y comportamiento esperado
- 15. Diagrama de flujo

1. Descripción general

El sistema de filtros permite al usuario encontrar bandas según dos criterios combinables:

Filtro	Tipo	Descripción
Género musical	Multisección (checkboxes)	Filtra bandas que tengan al menos uno de los géneros seleccionados
Proximidad geográfica	Campo de texto + slider	Filtra bandas cuya sede esté dentro de un radio (km) respecto a una ubicación introducida

Los dos filtros son **acumulativos**: si se activan ambos, el resultado devuelve únicamente las bandas que cumplan los dos a la vez.

La búsqueda es **sin estado en servidor**: todos los parámetros viajan en la URL mediante una petición **GET**, lo que permite compartir o guardar en favoritos cualquier búsqueda.

2. Arquitectura y tecnologías implicadas

Navegador (Frontend)

Twig

..... templates/banda/index.html.twig

Renderiza el formulario de filtros y el grid de tarjetas

```

├── CSS ..... public/css/perfil.css (compartido con músicos)
│   Estilos del sidebar, checkboxes, slider y botón de geolocalización
├── JavaScript (inline)
│   · Slider de radio: actualiza el degradado y la etiqueta en tiempo real
│   · Slider deshabilitado: sincroniza el estado con el campo de ciudad
│   · Geolocalización: Geolocation API → reverse geocoding → rellena el
formulario
├── Stimulus (assets/controllers/google-places-autocomplete_controller.js)
│   Carga la API de Google Places y conecta el autocomplete al campo de texto
│   Comparte el mismo controlador con la vista de músicos

Servidor (Backend)
├── BandaController · src/Controller/BandaController.php
│   Recibe los parámetros GET, los parsea y decide qué query ejecutar
├── BandaRepository · src/Repository/BandaRepository.php
│   Consulta a la base de datos (géneros) y aplica el filtro de distancia
│   en PHP mediante la fórmula de Haversine

```

Stack: Symfony 7, Doctrine ORM, Twig, Stimulus (Symfony UX), Google Maps JavaScript API (Places + Geocoder), API de Geolocalización del navegador (W3C).

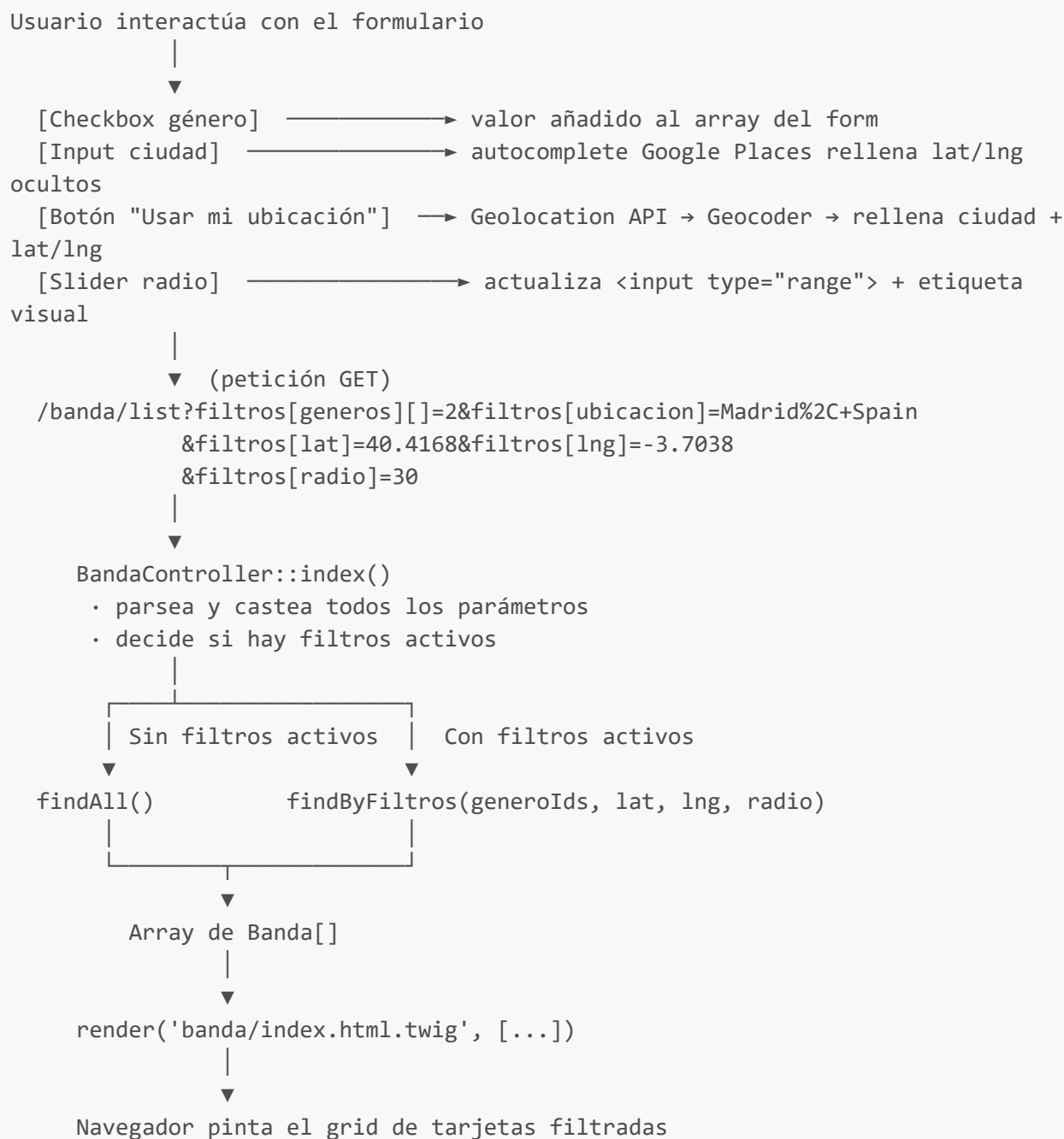
3. Diferencias respecto al sistema de filtros de músicos

El sistema de filtros de bandas es intencionadamente paralelo al de músicos ([docs/filtros-musicos.md](#)), con una diferencia principal:

Aspecto	Músicos	Bandas
Filtro de género	Sí (generosMusicales)	Sí (generosMusicales)
Filtro de instrumento	Sí	No — las bandas no tienen una relación de instrumentos filtrable
Filtro de proximidad	Sí	Sí
Distancia en tarjeta	Sí	Sí
Slider + geolocalización	Sí	Sí
Campo de texto visible	anyosExperiencia	anyoFormacion

La entidad [Banda](#) tiene campos [latitud](#) y [longitud](#) (igual que [Musico](#)) y una colección [generosMusicales](#) como relación [ManyToMany](#) con la entidad [Genero](#). Todo el código de filtrado sigue exactamente el mismo patrón.

4. Flujo completo de datos



5. Capa de presentación — Twig

Archivo: `templates/banda/index.html.twig`

El formulario usa `method="get"` y como `action` la ruta `app_banda_index`. Al no ser `POST`, los datos viajan directamente en la URL y no se necesita ningún token CSRF.

5.1 Layout

La plantilla usa la misma estructura `musicos-layout` que la vista de músicos: un `<aside>` de filtros a la izquierda y el grid de tarjetas a la derecha. Todos los estilos están definidos en `perfil.css`, que es compartido entre ambas vistas.

```

musicos-wrapper
├── musicos-header
├── musicos-layout
│   ├── musicos-filtros-sidebar (aside)
│   │   └── form[method=get]
│   │       ├── filtros-grupo (géneros)
│   │       └── filtros-grupo (ubicación + slider)
│   └── musicos-grid
│       └── musico-card × N

```

5.2 Checkboxes de géneros

```

<input type="checkbox"
      name="filtros[generos][]"
      value="{ { genero.id } }"
      {% if genero.id in (filtros.generos ?? []) %}checked{% endif %}>

```

El nombre `filtros[generos][]` hace que PHP lo interprete como un array: `$_GET['filtros']['generos'] = [1, 3]`. La condición `in (filtros.generos ?? [])` restaura el estado marcado al recargar la página con filtros activos.

5.3 Campo de ubicación

```

<div data-controller="google-places-autocomplete"
      data-google-places-autocomplete-api-key-value="{ { google_places_api_key } }">

  <input class="filtros-ubicacion-input" type="text" ...>    {# visible, solo UI
#}

  <button type="button" class="filtros-geolocate-btn" ...>  {# geolocalización
#}

  <input type="hidden" name="filtros[ubicacion]" ...>        {# nombre legible
#}

  <input type="hidden" name="filtros[lat]"
        data-google-places-autocomplete-target="lat">
  <input type="hidden" name="filtros[lng]"
        data-google-places-autocomplete-target="lng">
</div>

```

El campo de texto visible **no** tiene `name`. Solo viajan al servidor los tres `hidden`: texto de ubicación (para repopular el campo), latitud y longitud decimales.

5.4 Slider de radio

```
<input type="range" name="filtros[radio]"
      min="10" max="100" step="10"
      value="{{ filtros.radio ?? '20' }}">
```

- Rango: 10–100 km en pasos de 10. Valor por defecto: **20 km**.
- El atributo `value` restaura la posición del slider al recargar con filtros activos.
- El slider está **deshabilitado** si el campo de ciudad está vacío (ver sección 8.2).

5.5 Distancia en tarjeta

```
{% if banda.id in distancias|keys %}
    <span class="musico-card-distancia-tag">{{ distancias[banda.id] }} km</span>
{% endif %}
```

El span se inserta dentro de `.musico-card-tags`, heredando el separador `•` automático. Solo aparece cuando el filtro de proximidad está activo.

5.6 Géneros en tarjeta

```
{% if banda.generosMusicales|length > 0 %}
<div class="musico-card-instrumentos">
    {% for genero in banda.generosMusicales|slice(0, 3) %}
        <span class="musico-card-instrumento-tag">{{ genero.nombre }}</span>
    {% endfor %}
    {% if banda.generosMusicales|length > 3 %}
        <span class="musico-card-instrumento-tag">+{{
banda.generosMusicales|length - 3 }}</span>
    {% endif %}
</div>
{% endif %}
```

Se usan los géneros de la colección `generosMusicales` (relación ManyToMany) en lugar del campo string legacy `generos`, para ser coherentes con lo que filtra el backend. Se muestran máximo 3 pastillas; el excedente se aglutina en `+N`.

6. Autocompletado de ciudad — Stimulus + Google Places

Archivo: `assets/controllers/google-places-autocomplete_controller.js`

El controlador Stimulus es **compartido** con la vista de músicos. Se conecta al elemento que tenga `data-controller="google-places-autocomplete"`, por lo que funciona en ambas vistas sin modificación.

Ciclo de vida

```
connect() → ¿Google Maps ya cargado?
          | Sí → initAutocomplete()
          | No → loadScript() → [script cargado] → initAutocomplete()
```

El script de Google Maps (`libraries=places&language=es`) se carga una sola vez aunque haya múltiples instancias del controlador en la misma página, gracias a la comprobación de `id="google-maps-script"`.

Evento sintético tras selección

Después de escribir el valor en el `<input>` visible, el controlador dispara manualmente un evento `input`:

```
input.dispatchEvent(new Event('input'));
```

Esto es necesario para que la función `syncSlider()` (definida en el inline script de la plantilla) detecte el cambio y habilite el slider de radio.

7. Geolocalización del navegador

Código: bloque `<script>` al final de `templates/banda/index.html.twig`

El botón **"Usar mi ubicación"** usa la Geolocation API (W3C). Solo funciona en HTTPS o localhost.

Flujo

```

Clic en el botón
  |
  ▼
navigator.geolocation.getCurrentPosition(éxito, error)
  |
  ▼ (éxito)
pos.coords.latitude / pos.coords.longitude
  |
  ├── Rellena filtros[lat] y filtros{lng]
  |
  └── ¿window.google disponible?
      | Sí → Geocoder().geocode() → "Ciudad, País"
      | No → "lat, lng" literal
  |
  ▼
textInput.dispatchEvent(new Event('input')) → activa syncSlider()
  |
  ▼ (error – permiso denegado)
Botón vuelve a estado original

```

Por qué se necesita el reverse geocoding

El backend no usa `filtros[ubicacion]` para calcular distancias; solo necesita `lat` y `lng`. El texto de ciudad es puramente visual para que el campo muestre algo legible al recargar la página con los filtros activos.

8. Slider de radio de proximidad

8.1 Degradado visual

```
function updateSlider(v) {  
    var pct = (v - 10) / 90 * 100;  
    slider.style.background =  
        'linear-gradient(to right, #9b30ff ' + pct + '%, rgba(155,48,255,0.18) ' +  
pct + '%)';  
    label.textContent = v + ' km';  
}
```

- Rango útil: 90 unidades (10 a 100).
- Fórmula: $(v - 10) / 90 \times 100$ normaliza el valor a un porcentaje 0–100%.
- El degradado divide la barra en zona activa (púrpura) y zona inactiva (púrpura tenue).

8.2 Slider deshabilitado sin ubicación

El slider no tiene efecto si no hay coordenadas. Para evitar que el usuario lo mueva sin resultado:

```
function syncSlider() {  
    var hasLocation = ubicacionInput.value.trim() !== '';  
    slider.disabled = !hasLocation;  
    slider.closest('.filtros-radio-wrap')  
        .classList.toggle('filtros-radio-disabled', !hasLocation);  
    if (!hasLocation) {  
        slider.style.background = 'rgba(155,48,255,0.18)';  
    } else {  
        updateSlider(+slider.value);  
    }  
}  
ubicacionInput.addEventListener('input', syncSlider);  
syncSlider(); // estado inicial en carga de página
```

`syncSlider()` se ejecuta al cargar la página (para restaurar el estado si hay ubicación en la URL) y cada vez que cambia el campo de ciudad, sea por tipado manual, por Google Places o por el botón de geolocalización.

Cuando `slider.disabled = true`, el navegador **no envía** el campo en el formulario GET, lo que es correcto: sin ubicación el backend ignoraría el radio de todos modos.

9. Distancia en la tarjeta de banda

Diseño

Para no alterar la firma de `findByFiltros()`, las distancias se calculan en el controlador como un **array paralelo** al array de bandas, indexado por ID:

```
findByFiltros()           → Banda[]           (sin cambios)
calcularDistanciaKm() × N → float[] $distancias (paralelo, nuevo)
```

Código del controlador

```
$distancias = [];
if ($lat !== null && $lng !== null) {
    foreach ($bandas as $b) {
        if ($b->getLatitud() !== null && $b->getLongitud() !== null) {
            $distancias[$b->getId()] = round(
                $bandaRepository->calcularDistanciaKm($lat, $lng, $b-
>getLatitud(), $b->getLongitud()),
                1
            );
        }
    }
}
```

- Las bandas sin coordenadas no tienen entrada en `$distancias` y su tarjeta no mostrará distancia.
- Cuando no hay filtro de ubicación, `$distancias` llega vacío a la plantilla.

10. Capa de control — BandaController

Archivo: `src/Controller/BandaController.php` **Ruta:** `GET /banda/list` → `app_banda_index`

Parseo de parámetros

```
$filtros = $request->query->all('filtros');

$generoIds = !empty($filtros['generos']) ? array_map('intval', (array)
    $filtros['generos']) : [];
$lat = isset($filtros['lat']) && $filtros['lat'] !== '' ? (float)
    $filtros['lat'] : null;
$lng = isset($filtros['lng']) && $filtros['lng'] !== '' ? (float)
    $filtros['lng'] : null;
$radio = isset($filtros['radio']) && $filtros['radio'] !== '' ? (int)
    $filtros['radio'] : null;
```

Cada campo ausente o vacío resulta en `null` / array vacío, nunca en un error.

Decisión de consulta


```
$hayFiltros = !empty($generoIds) || ($lat !== null && $radio !== null);

$bandas = $hayFiltros
    ? $bandaRepository->findByFiltros($generoIds, $lat, $lng, $radio)
    : $bandaRepository->findAll();
```

Sin filtros activos se usa `findAll()` directamente, evitando la query más costosa.

Datos enviados a la plantilla

```
return $this->render('banda/index.html.twig', [
    'bandas'      => $bandas,           // resultado filtrado
    'generos'     => $generoRepository->findBy([], ['nombre' => 'ASC']),
    'filtros'     => $filtros,         // para repopular el formulario
    'distancias' => $distancias,       // array id → km, vacío si no hay ubicación
]);
```

`GeneroRepository` es el mismo que usa `MusicoController`; los géneros son compartidos entre músicos y bandas.

11. Capa de acceso a datos — BandaRepository

Archivo: `src/Repository/BandaRepository.php`

`findByFiltros()`

```
public function findByFiltros(array $generoIds, ?float $lat, ?float $lng, ?int
$radio): array
```

Fase 1 — Filtro en base de datos

```
$qb = $this->createQueryBuilder('b');

if (!empty($generoIds)) {
    $qb->join('b.generosMusicales', 'g')
        ->andWhere('g.id IN (:generos)')
        ->setParameter('generos', $generoIds);
}

$bandas = $qb->distinct()->getQuery()->getResult();
```

El JOIN sobre `generosMusicales` utiliza la tabla intermedia `banda_genero` definida en la entidad con `# [ORM\JoinTable(name: 'banda_genero')]`. `distinct()` evita duplicados cuando una banda tiene varios

géneros coincidentes.

Fase 2 — Filtro geográfico en PHP

```
if ($lat !== null && $lng !== null && $radio !== null && $radio > 0) {
    $bandas = array_values(array_filter($bandas, function (Banda $b) use ($lat,
$lng, $radio) {
        if ($b->getLatitud() === null || $b->getLongitud() === null) {
            return false;
        }
        return $this->haversine($lat, $lng, $b->getLatitud(), $b->getLongitud())
<= $radio;
    }));
}
```

Las bandas sin coordenadas guardadas son excluidas cuando el filtro geográfico está activo.

calcularDistanciaKm()

Método público que expone `haversine()` para uso desde el controlador, sin duplicar la fórmula:

```
public function calcularDistanciaKm(float $lat1, float $lng1, float $lat2, float
$lng2): float
{
    return $this->haversine($lat1, $lng1, $lat2, $lng2);
}
```

12. Fórmula de Haversine

Calcula la distancia ortodrómica entre dos puntos geográficos sobre la superficie esférica de la Tierra.

```
private function haversine(float $lat1, float $lng1, float $lat2, float $lng2):
float
{
    $dLat = deg2rad($lat2 - $lat1);
    $dLng = deg2rad($lng2 - $lng1);
    $a = sin($dLat / 2) ** 2
        + cos(deg2rad($lat1)) * cos(deg2rad($lat2)) * sin($dLng / 2) ** 2;
    return 6371 * 2 * atan2(sqrt($a), sqrt(1 - $a));
}
```

Variable	Significado
<code>\$dLat</code> , <code>\$dLng</code>	Diferencias de latitud y longitud en radianes

Variable	Significado
a	Cuadrado del semiseno del ángulo central
6371	Radio medio de la Tierra en km
Resultado	Distancia en kilómetros

$$d = 2R \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}) \quad \text{donde} \quad a = \sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}$$

El error es inferior al 0,3% para distancias terrestres normales, más que suficiente para un radio de búsqueda de decenas de km.

Esta implementación es **idéntica** a la de [MusicoRepository](#). Cada repositorio tiene su propia copia privada para mantener la independencia entre entidades.

13. Estructura de parámetros GET

URL de ejemplo con todos los filtros activos:

```
/banda/list
?filtros[generos][]=2
&filtros[generos][]=5
&filtros[ubicacion]=Madrid%2C+Spain
&filtros[lat]=40.4168
&filtros[lng]=-3.7038
&filtros[radio]=30
```

Parámetro	Tipo PHP	Uso
<code>filtros[generos][]</code>	<code>int[]</code>	IDs de géneros a filtrar (puede repetirse)
<code>filtros[ubicacion]</code>	<code>string</code>	Texto legible de la ciudad (solo visual)
<code>filtros[lat]</code>	<code>float</code>	Latitud del punto de referencia
<code>filtros[lng]</code>	<code>float</code>	Longitud del punto de referencia
<code>filtros[radio]</code>	<code>int</code>	Radio en km (10–100, pasos de 10)

Nota: no existe `filtros[instrumentos][]` porque las bandas no tienen filtro por instrumento.

14. Casos de uso y comportamiento esperado

Solo géneros seleccionados

Query con `JOIN generosMusicales` y `WHERE g.id IN (...)`. No se aplica filtro geográfico. Se devuelven todas las bandas con al menos uno de esos géneros.

Solo ubicación + radio

La query base recupera todas las bandas y luego se filtra en PHP por distancia Haversine. Las bandas sin coordenadas son excluidas.

Género + ubicación + radio

Query con **JOIN** → resultado → filtrado adicional por Haversine. El resultado son bandas que tengan ese género Y estén dentro del radio indicado.

Sin ningún filtro

Se llama directamente a **findAll()**, evitando la query más costosa.

Ubicación sin radio (o radio sin ubicación)

El filtro geográfico no se activa. La comprobación **\$lat !== null && \$radio !== null** exige ambos. El slider además está visualmente deshabilitado si el campo de ciudad está vacío.

Banda sin coordenadas

Si una banda tiene **latitud/longitud** a **null** en la base de datos, es excluida de los resultados cuando hay filtro geográfico activo.

15. Diagrama de flujo



| Sí → findByFiltros(...)

BandaRepository

QueryBuilder
JOIN géneros
DISTINCT

Banda[]

▼
¿lat + radio activos?

| Sí → array_filter Haversine

| No → resultado tal cual

calcularDistanciaKm() × N
→ \$distancias[id => km]

render('banda/index.html.twig')

▼ HTML con grid de tarjetas filtradas

NAVEGADOR (respuesta)

- Grid muestra solo las bandas que pasan todos los filtros
- Checkboxes y slider restaurados con los valores de la URL (repopulación desde \$filtros)
- Distancia mostrada en tarjeta si hay filtro de proximidad activo